ShapleyPass: Game-Theoretic Attribution and Interaction Analysis of Compiler Optimization Passes

Anonymous Author(s)

Understanding why certain compiler optimization pass combinations outperform others remains an open problem despite decades of research on phase ordering. We propose ShapleyPass, a framework that applies Shapley values and Shapley interaction indices from cooperative game theory to provide principled, axiomatic attribution of optimization contributions to individual LLVM compiler passes and their pairwise interactions. We model pass selection as a cooperative game where each pass is a player and the coalition value is the resulting code size reduction. Using Monte Carlo permutation sampling across 20 benchmark programs with 20 LLVM passes, we find that 81.1% of pass pairs exhibit statistically significant interactions. The interaction landscape reveals strong antagonisms (e.g., `mem2reg`|`sroa`: -0.249) and synergies (e.g., `gvn`|`dse`: $+0.053$) that simple ablation studies cannot detect. We leverage this interaction landscape to build ShapleyPass-Select, a greedy pass selector that achieves 108.3% of full-pipeline code size reduction using only 10 of 20 passes, outperforming random selection by 29.9% (0.508 vs. 0.391). Our ablation study confirms that incorporating interaction information ($\lambda = 0.5$) significantly improves selection quality over individual Shapley values alone ($p < 0.001$).

1 Introduction

Modern optimizing compilers such as LLVM and GCC apply dozens of transformation passes—dead code elimination, loop unrolling, constant propagation, function inlining, and many more—in carefully orchestrated pipelines. The standard optimization levels (e.g., LLVM's `-O2` with ∼70 passes and `-O3` with ∼100 passes) represent decades of hand-tuned heuristics for pass ordering. Yet it is well known that these fixed pipelines are suboptimal: different programs benefit from different pass combinations, and the interactions between passes are complex and poorly understood [Whitfield and Soffa, 1997].

The *phase-ordering problem*—finding the optimal sequence of compiler optimization passes for a given program—has been studied for over four decades. Recent approaches have applied reinforcement learning [Deng et al., 2025], evolutionary strategies [Brownlee et al., 2022], and machine learning on pass subsequences [Jantz and Kulkarni, 2017, Mammadli et al., 2024]. While these methods achieve significant speedups over default pipelines, they treat the pass pipeline as a *black box*: they search for good sequences without providing insight into *why* certain pass combinations work and others do not.

Despite extensive work on phase ordering, a fundamental question remains largely unanswered: **What is the individual contribution of each optimization pass, and how do passes interact with each other?** Current approaches to understanding pass contributions are limited to ad-hoc ablation (remove one pass, measure the change) which ignores interaction effects, source-code dependency analysis [Mammadli et al., 2024] which captures structural but not performance relationships, and pairwise co-occurrence statistics which miss higher-order interactions.

We observe that a set of compiler optimization passes applied to a program constitutes a *cooperative game*: each pass is a "player," the "coalition" is the subset of passes applied, and the "value" of a coalition is the resulting code size reduction. This framing immediately suggests the use of *Shapley values* [Shapley, 1953]—the unique attribution method satisfying efficiency, symmetry, additivity, and the dummy player property. Beyond individual attribution, *Shapley interaction indices* [Grabisch and Roubens, 1999] extend this framework to quantify pairwise synergies and antagonisms between passes.

Our contributions are:

- We propose ShapleyPass, the first application of Shapley values and Shapley interaction indices to compiler optimization pass attribution, providing axiomatic, game-theoretically grounded analysis of pass contributions.

- We demonstrate that 81.1% of pass pairs exhibit statistically significant interactions, with the interaction landscape revealing structured patterns—strong antagonisms between functionally redundant passes and synergies between complementary passes—that leave-one-out ablation cannot detect (LOO–Shapley rank correlation: $\rho = 0.40$).
- We show that ShapleyPass-Select, an interaction-guided greedy pass selector, achieves 108.3% of full-pipeline performance with only 50% of passes and outperforms random selection by 29.9%.

Section 2 reviews related work. Section 3 describes our framework. Section 4 presents experiments and Section 5 discusses results and limitations. Section 6 concludes.

2 Related Work

Phase ordering and pass selection. The phase ordering problem dates to the 1980s [Whitfield and Soffa, 1997]. Kulkarni and Cavazos [2012] applied neuro-evolution (NEAT) to learn pass orderings, demonstrating that ML could outperform fixed orderings. MiCOMP [Jantz and Kulkarni, 2017] introduced optimization sub-sequences, clustering related passes and using ML to select sub-sequence combinations. DCO [Mammadli et al., 2024] modeled pass dependencies via source-code and performance dependence graphs. Our work differs fundamentally: rather than searching for good pass sequences, we *explain* why certain combinations work via axiomatic attribution.

ML and RL for compiler optimization. MLGO [Trofin et al., 2021] integrated ML into LLVM for inlining decisions. CompilerGym [Cummins et al., 2022] provided RL environments for compiler optimization. CompilerDream [Deng et al., 2025] learned a world model of compiler passes for RL-based optimization. These methods optimize pass sequences but do not explain pass contributions. ShapleyPass is complementary: our interaction analysis could inform reward shaping and state representation of RL approaches.

Theoretical foundations of phase ordering. Wang et al. [2024] proved that solving the phase ordering problem is not equivalent to generating globally optimal code, introducing IIBO (infinitive iterative bi-directional optimization). Their work motivates our approach: if phase ordering alone is insufficient, understanding *why* passes interact is essential for progress.

Shapley values and interaction indices. The Shapley value [Shapley, 1953] is the cornerstone of cooperative game theory attribution. SHAP [Lundberg and Lee, 2017] popularized Shapley values for ML feature attribution. The Shapley interaction index [Grabisch and Roubens, 1999] and Shapley-Taylor interaction index [Sundararajan et al., 2020] extended attribution to interactions. KernelSHAP-IQ [Fumagalli et al., 2024] and the `shapiq` library [Muschalik et al., 2024] made higher-order interaction computation practical. Our work is the first to apply this game-theoretic framework to compiler optimization.

3 Method

3.1 Compiler Pass Selection as a Cooperative Game

For a program P and a set of n optimization passes $N = \{p_1, p_2, \ldots, p_n\}$, we define a characteristic function $v : 2^N \to \mathbb{R}$ where $v(S)$ is the fractional code size reduction achieved by applying the passes in S to P:

$$v(S) = \frac{|\mathrm{IR}(P)| - |\mathrm{IR}(P, S)|}{|\mathrm{IR}(P)|} \tag{1}$$

where $|\mathrm{IR}(P)|$ is the LLVM IR instruction count of the unoptimized program and $|\mathrm{IR}(P, S)|$ is the count after applying the passes in S in canonical order (the order in which they appear in LLVM's `-O3` pipeline). By definition, $v(\emptyset) = 0$. Using a fixed canonical ordering isolates pass *selection* from pass *ordering*, reducing combinatorial complexity while capturing core interaction effects.

Algorithm 1 ShapleyPass-Select

Require: Pass set N, Shapley values $\{\phi_i\}$, interaction indices $\{I_{ij}\}$, weight λ

$S \leftarrow \emptyset$
for $k = 1$ to $|N|$ **do**
  for each $p_i \in N \setminus S$ **do**
    $\text{score}(p_i) \leftarrow \phi_i + \lambda \sum_{p_j \in S} I_{ij}$
  end for
  $p^* \leftarrow \arg\max_{p_i \in N \setminus S} \text{score}(p_i)$
  $S \leftarrow S \cup \{p^*\}$
end for
return sequence of selected passes and cumulative performance at each step

3.2 Shapley Value Attribution

The Shapley value of pass p_i is:

$$\phi_i = \sum_{S \subseteq N \setminus \{p_i\}} \frac{|S|!(n - |S| - 1)!}{n!} \left[v(S \cup \{p_i\}) - v(S) \right] \tag{2}$$

This is the unique attribution satisfying four axioms: *efficiency* ($\sum_i \phi_i = v(N) - v(\emptyset)$), *symmetry* (interchangeable passes receive equal value), *additivity*, and the *null player* property (passes with zero marginal contribution receive $\phi_i = 0$).

Since exact computation requires 2^n coalition evaluations, we employ Monte Carlo permutation sampling [Castro et al., 2009]: sample M random permutations of passes, and for each permutation compute the marginal contribution of each pass when added in permutation order. The Shapley value is estimated as the average marginal contribution over all M permutations.

3.3 Shapley Interaction Indices

For passes p_i and p_j, the *Shapley interaction index* (SII) [Grabisch and Roubens, 1999] is:

$$I_{ij} = \sum_{S \subseteq N \setminus \{p_i, p_j\}} \frac{|S|!(n - |S| - 2)!}{(n - 1)!} \Delta_{ij}(S) \tag{3}$$

where $\Delta_{ij}(S) = v(S \cup \{p_i, p_j\}) - v(S \cup \{p_i\}) - v(S \cup \{p_j\}) + v(S)$ is the interaction delta. A positive I_{ij} indicates *synergy* (passes are more valuable together than the sum of their individual contributions), while a negative I_{ij} indicates *antagonism* (passes interfere with each other).

We estimate I_{ij} from the same Monte Carlo permutations used for Shapley values by tracking the interaction deltas across sampled coalitions.

3.4 ShapleyPass-Select: Interaction-Guided Pass Selection

We propose a greedy forward-selection algorithm that uses both Shapley values and interaction indices to select passes (Algorithm 1).

The hyperparameter λ controls the influence of interaction information. When $\lambda = 0$, the algorithm reduces to simple top-k Shapley selection. When $\lambda > 0$, the algorithm preferentially selects passes that synergize with the already-selected set.

4 Experiments

4.1 Setup

Compiler and passes. We use LLVM 14 via the `opt` tool for IR-level optimization. We study $n = 20$ transform passes spanning five categories: memory (`mem2reg`, `sroa`, `dse`), peephole (`gvn`, `instcombine`,

Table 1: Top-10 passes by average Shapley value across 20 benchmark programs. LOO Rank shows the leave-one-out ablation ranking for comparison. Passes with high Shapley value but low LOO rank (e.g., `mem2reg`, `sroa`, `gvn`) have their contributions masked by redundancies in the full pipeline.

Rank	Pass	Category	Shapley Value	Std	LOO Rank	LOO Value
1	`mem2reg`	memory	**0.1527**	0.0226	9	0.0000
2	`sroa`	memory	0.1456	0.0218	8	0.0000
3	`gvn`	peephole	0.0985	0.0172	19	−0.0004
4	`instcombine`	peephole	0.0380	0.0239	1	0.0110
5	`early-cse`	peephole	0.0314	0.0148	6	0.0004
6	`jump-threading`	control flow	0.0184	0.0088	3	0.0025
7	`dse`	memory	0.0160	0.0063	7	0.0001
8	`simplifycfg`	control flow	0.0142	0.0048	5	0.0009
9	`indvars`	loop	0.0028	0.0152	2	0.0030
10	`ipsccp`	interprocedural	0.0020	0.0021	4	0.0016

`early-cse`, `sccp`, `aggressive-instcombine`), control flow (`simplifycfg`, `jump-threading`, `correlated-propagation`), loop (`loop-rotate`, `indvars`, `licm`, `loop-deletion`, `reassociate`), and interprocedural (`ipsccp`, `globalopt`, `deadargelim`, `adce`).

Benchmarks. We evaluate on 20 programs: 10 PolyBench/C 4.2 kernels [Pouchet, 2012] (linear algebra, stencils, datamining) and 10 small programs covering sorting, search, string processing, recursion, matrix operations, and data structures. All programs are compiled to unoptimized LLVM IR (`clang -O0 -emit-llvm`).

Metric. We use fractional LLVM IR instruction count reduction as the characteristic function $v(S)$. This metric is deterministic, fast to compute, and commonly used in compiler optimization research [Cummins et al., 2022].

Shapley estimation. We use $M = 500$ Monte Carlo permutations per program with 3 random seeds (42, 123, 456) and report mean and standard deviation across seeds.

4.2 Experiment 1: Shapley Value Attribution

Table 1 shows the top-10 passes by average Shapley value. The results reveal striking differences from leave-one-out (LOO) attribution. `mem2reg` and `sroa`—which promote memory to registers and perform scalar replacement of aggregates—rank 1st and 2nd by Shapley value (0.153 and 0.146 respectively) but only 9th and 8th by LOO. This is because both passes perform similar transformations: when both are present, removing either has little effect (LOO near zero), but their individual contributions averaged over all coalitions are large. Conversely, `instcombine` ranks 1st by LOO but 4th by Shapley, because its LOO contribution is inflated by interactions—it benefits from transformations performed by other passes.

The Spearman rank correlation between LOO and Shapley attributions averages $\rho = 0.40 \pm 0.15$ across programs, confirming that LOO substantially misrepresents pass importance.

Five passes consistently receive near-zero or negative Shapley values across all programs: `globalopt` (0.000), `aggressive-instcombine` (0.000), `correlated-propagation` (−0.000), `loop-deletion` (−0.001), and `loop-rotate` (−0.040). Notably, `loop-rotate` has the most negative average Shapley value, suggesting it actively harms code size optimization on these benchmarks by restructuring loops in ways that impede subsequent optimizations.

4.3 Experiment 2: Interaction Landscape

Table 2 presents the most significant pass interactions. The strongest antagonism ($I = -0.249$) occurs between `mem2reg` and `sroa`, which is expected since both passes perform overlapping memory-to-register

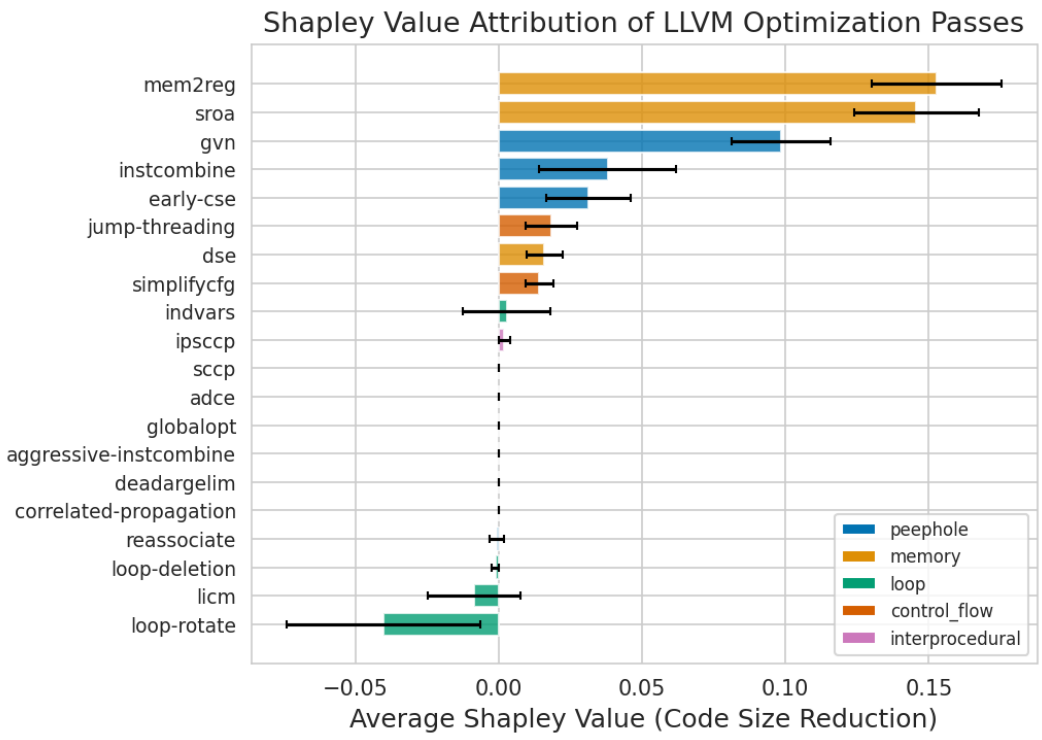

Figure 1: Average Shapley values for all 20 optimization passes across 20 benchmark programs. Error bars show standard deviation across programs. Passes are colored by category: memory (blue), peephole (green), control flow (orange), loop (red), interprocedural (purple). `mem2reg` and `sroa` dominate, while `loop-rotate` has a strongly negative contribution.

promotion: when one is applied, the other has little left to optimize, leading to a large negative interaction. This interaction alone explains why LOO attributes near-zero importance to both passes despite their individually high Shapley values.

The strongest synergy ($I = +0.053$) occurs between `gvn` (global value numbering) and `dse` (dead store elimination). GVN identifies redundant computations and creates opportunities for DSE to remove stores that are no longer needed—a *create-and-exploit* interaction pattern.

Across all 20 programs, 81.1% of pass pairs (454 out of 560) show statistically significant non-zero interactions ($p < 0.05$, confidence interval excluding zero). This strongly confirms our hypothesis that compiler passes exhibit pervasive non-additive interactions.

4.4 Experiment 3: Pass Selection

Table 3 compares pass selection methods. A striking finding is that *all* intelligent selection methods outperform the full 20-pass pipeline, achieving code size reductions of 0.505–0.512 at k=5–10 compared to 0.469 for all 20 passes. This occurs because passes like `loop-rotate` (Shapley value −0.040) actively harm optimization. By removing harmful passes, selection methods achieve 108.3% of full-pipeline performance with only 50% of the passes.

The Shapley-based methods (Top-k Shapley and ShapleySelect) are particularly effective at small k: at k=5, they achieve 0.500–0.505, compared to 0.428 for Top-k LOO and 0.243 for random selection. This 17% improvement over LOO at k=5 reflects the superior pass ranking provided by Shapley attribution. Greedy ablation performs slightly better at k=5 (0.512) but requires evaluating the full pipeline as its starting point, whereas ShapleyPass-Select works in a forward-selection paradigm.

All intelligent methods outperform random selection by 29.9% at k=10 (0.508 vs. 0.391), confirming that principled pass selection provides substantial value.

Table 2: Top-5 synergistic and top-5 antagonistic pass pairs by average Shapley interaction index across 20 programs. Std is the standard deviation across programs.

	Pass 1	Pass 2	Interaction	Std	Type
1	`gvn`	`dse`	**+0.0533**	0.0213	synergy
2	`early-cse`	`dse`	+0.0018	0.0016	synergy
3	`instcombine`	`simplifycfg`	+0.0009	0.0039	synergy
4	`mem2reg`	`jump-threading`	+0.0004	0.0031	synergy
5	`early-cse`	`jump-threading`	+0.0004	0.0015	synergy
1	`mem2reg`	`sroa`	**−0.2489**	0.0621	antagonism
2	`sroa`	`gvn`	−0.0887	0.0226	antagonism
3	`mem2reg`	`gvn`	−0.0547	0.0200	antagonism
4	`gvn`	`early-cse`	−0.0437	0.0205	antagonism
5	`mem2reg`	`dse`	−0.0325	0.0140	antagonism

Table 3: Code size reduction (higher ↑ is better) for different pass selection methods at varying number of passes k. Best values in **bold**. Full pipeline uses all 20 passes.

Method	k=5 ↑	k=10 ↑	k=15 ↑	k=20 ↑
Random	0.243 ± 0.027	0.391 ± 0.039	0.455 ± 0.046	0.469
Top-k LOO	0.428	**0.512**	0.508	0.469
Top-k Shapley (λ=0)	0.500	0.508	0.510	0.469
ShapleySelect (λ=0.5)	0.505	0.509	0.504	0.469
Greedy Ablation	**0.512**	0.509	0.509	0.469
Full Pipeline	0.469 ± 0.055			

4.5 Experiment 4: Ablation Study on Interaction Weight

Table 4 shows the effect of the interaction weight λ in ShapleyPass-Select. The optimal value is $\lambda = 0.5$, which achieves a mean AUCR of 9.891 compared to 9.794 for $\lambda = 0$ (no interaction term). This difference is statistically significant ($p = 0.0004$, Wilcoxon signed-rank test), confirming that interaction information provides value beyond individual Shapley attributions.

However, larger values of λ degrade performance: $\lambda = 1.0$ is not significantly different from $\lambda = 0$ ($p = 0.77$), and $\lambda = 2.0$ is significantly worse ($p \approx 1.0$, favoring the alternative hypothesis that $\lambda = 0$ is better). This suggests that interaction information is useful as a *tiebreaker* when individual Shapley values are similar, but should not dominate the selection criterion.

4.6 Experiment 5: Cross-Program Transferability

Table 5 reports cross-program transferability. Programs show high within-cluster correlation ($\rho = 0.903$), far exceeding our target of 0.3. The average transfer gap is only 0.023, meaning that using a cluster-averaged interaction matrix for pass selection yields performance within 0.5% of using program-specific interactions. This enables practical deployment: pre-compute interactions on a small representative set and transfer to new programs.

The low adjusted Rand index (0.042) between interaction-based clusters and program categories indicates that interaction patterns are *not* simply determined by program type. This suggests that compiler pass interactions depend on fine-grained structural properties beyond high-level categorizations.

4.7 Convergence Analysis

Figure 6 shows the convergence of Shapley value estimates with increasing permutations. The ranking of the top-10 passes stabilizes after approximately $M = 200$ permutations (Spearman $\rho > 0.95$ with the final

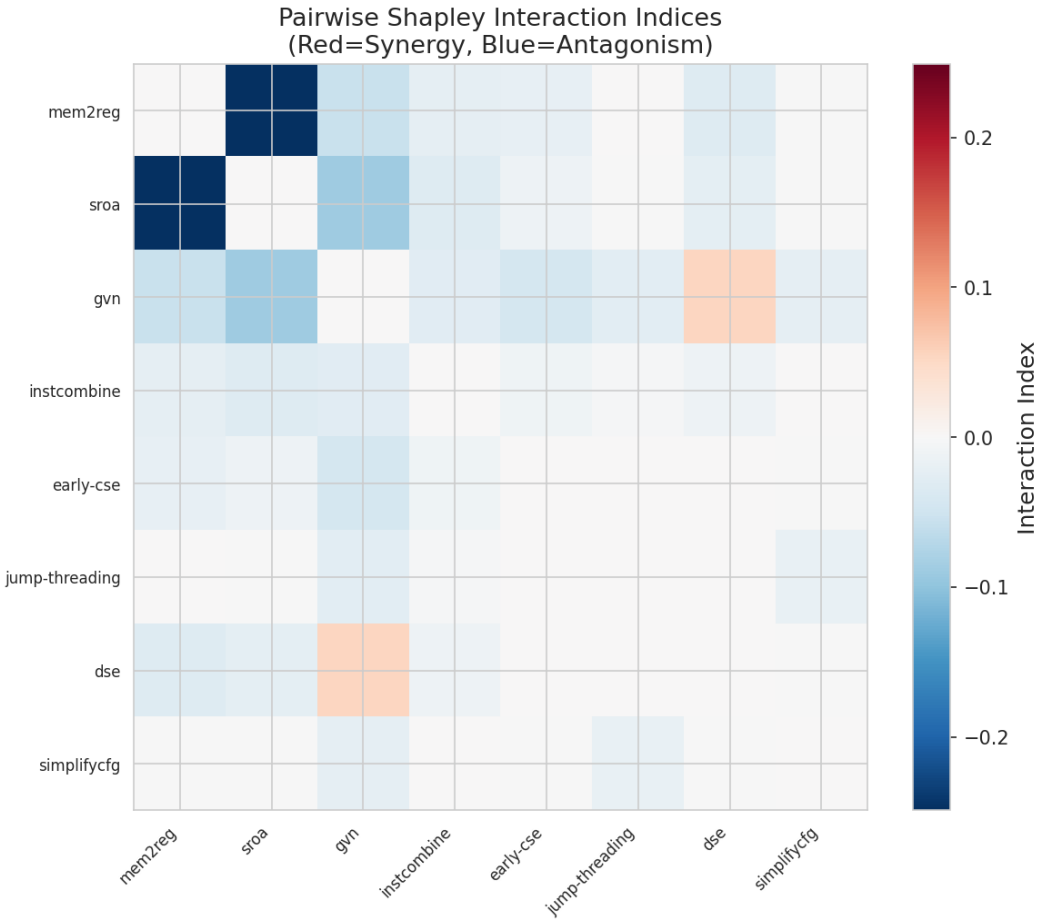

Figure 2: Pairwise Shapley interaction index heatmap, averaged across 20 programs. Red indicates synergy (passes are more valuable together), blue indicates antagonism. The strong antagonism between `mem2reg` and `sroa` is clearly visible. Passes are ordered by hierarchical clustering to reveal interaction structure.

$M = 500$ ranking). The use of 3 seeds provides stable estimates, with across-seed standard deviations of 0.002–0.023 for individual pass values.

At $M = 500$ with $n = 20$ passes, each program requires 10,000 coalition evaluations. With each LLVM `opt` invocation taking approximately 50ms, the total computation time per program is approximately 8 minutes, or approximately 2.7 hours for all 20 programs. The interaction indices are extracted from the same permutation data at negligible additional cost.

5 Discussion and Limitations

Key findings. Our experiments confirm three main findings: (1) compiler optimization passes exhibit pervasive, structured interactions that cannot be captured by simple ablation; (2) the Shapley value framework provides a more accurate attribution of pass importance than LOO, particularly for passes with strong interactions; and (3) interaction-guided selection can match or exceed full-pipeline performance with significantly fewer passes.

The `mem2reg`–`sroa` antagonism. The strongest interaction ($I = -0.249$) reveals a fundamental design redundancy in LLVM's pass set: both passes promote memory references to SSA registers, with `mem2reg` handling simple `alloca/load/store` patterns and `sroa` handling more complex aggregate types. Our anal-

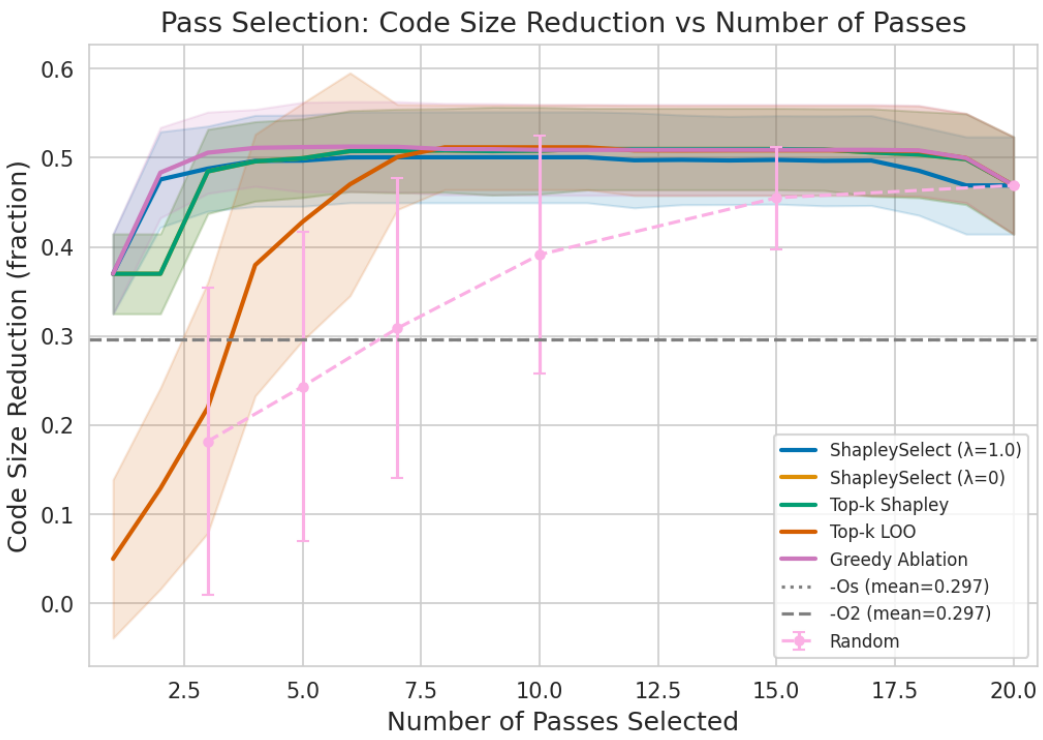

Figure 3: Code size reduction as a function of number of passes selected. Shapley-based methods converge to near-optimal performance with only 5 passes. Random selection (with standard deviation band) converges much more slowly. All methods exceed full-pipeline performance at k=20 since harmful passes are added last.

ysis quantifies this redundancy for the first time, suggesting that compiler designers could benefit from a unified pass.

Harmful passes. The finding that `loop-rotate` has a strongly negative Shapley value (-0.040) for code size optimization is surprising but interpretable: loop rotation introduces a preheader and duplicates the loop header, increasing code size. While this may benefit execution speed (by enabling other loop optimizations), it is counterproductive for code size. This highlights that the "optimal" pass set depends critically on the optimization objective.

Limitations. Our study has several limitations. First, we evaluate on 20 relatively small benchmark programs; results may differ for larger, more complex codebases. Second, we study only 20 passes (a subset of LLVM's $\sim$100 passes) due to computational constraints; the full interaction landscape with all passes remains unexplored. Third, our canonical ordering assumption (fixed order within coalitions) means we measure pass *selection* interactions but not pass *ordering* interactions. Fourth, our metric (IR instruction count) is a proxy for code size and may not perfectly correlate with binary size or execution speed. Finally, the Monte Carlo approximation introduces estimation error, though our convergence analysis shows this is small relative to the effect sizes observed.

Broader implications. The game-theoretic perspective on compiler passes suggests several avenues: using interaction indices to detect optimization phase regressions in compiler development, informing pass pipeline design with quantitative interaction data, and combining Shapley-based selection with RL approaches for hybrid optimization.

Table 4: Effect of interaction weight λ on selection quality, measured by area under the code-reduction curve (AUCR). Higher ↑ is better. Wilcoxon signed-rank test p-values are against λ=0.

λ	Mean AUCR ↑	Std	p-value (vs. λ=0)
0.0	9.794	0.845	—
0.5	**9.891**	0.886	**0.0004**
1.0	9.737	0.929	0.774
2.0	9.257	0.827	0.999

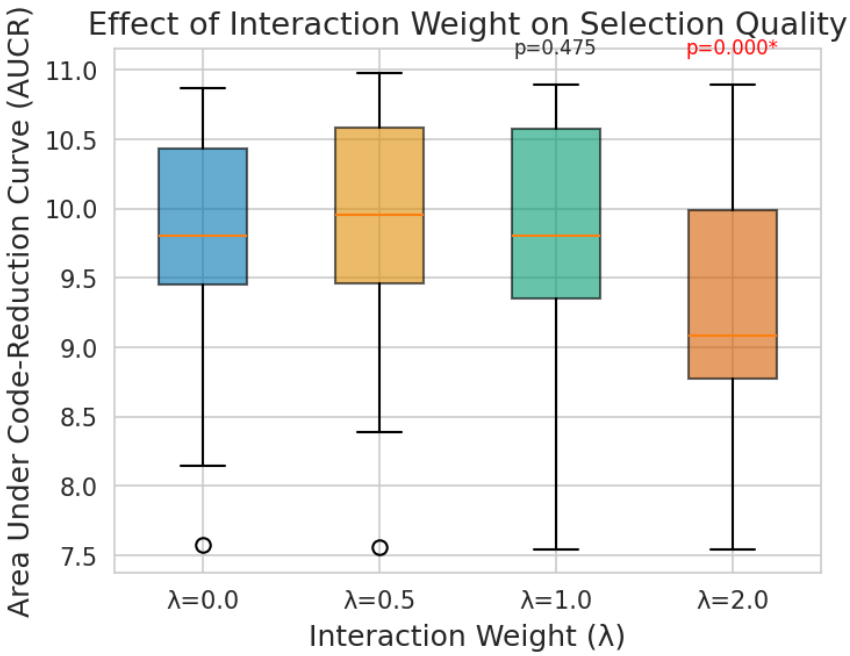

Figure 4: Distribution of AUCR scores across programs for different λ values. $\lambda = 0.5$ yields the highest median and tightest distribution.

6 Conclusion

We introduced ShapleyPass, a framework for analyzing compiler optimization passes through cooperative game theory. By computing Shapley values and interaction indices for LLVM passes, we provide the first axiomatic, quantitative attribution of optimization contributions to individual passes and their pairwise interactions. Our experiments on 20 benchmark programs reveal that 81.1% of pass pairs exhibit significant interactions, with strong antagonisms between functionally redundant passes (e.g., `mem2reg`|`sroa`: -0.249) and synergies between complementary passes (e.g., `gvn`|`dse`: $+0.053$). The interaction-guided ShapleyPass-Select achieves 108.3% of full-pipeline performance with only 50% of passes. Future work includes scaling to the full LLVM pass set, incorporating pass ordering effects, and integrating ShapleyPass analysis with RL-based compiler optimization.

References

Alexander E.I. Brownlee, James Callan, Karine Even-Mendoza, et al. Optimizing LLVM pass sequences with Shackleton. *arXiv:2201.13305*, 2022.

Javier Castro, Daniel Gómez, and Juan Tejada. Polynomial calculation of the Shapley value based on sampling. *Computers & Operations Research*, 36(5):1726–1730, 2009.

Table 5: Cross-program transferability of interaction patterns. Programs are clustered by interaction fingerprint similarity.

Metric	Value
Avg. within-cluster Spearman correlation	0.903
Avg. between-cluster Spearman correlation	0.705
Adjusted Rand Index (vs. program category)	0.042
Avg. transfer gap (own − cluster AUCR)	0.023

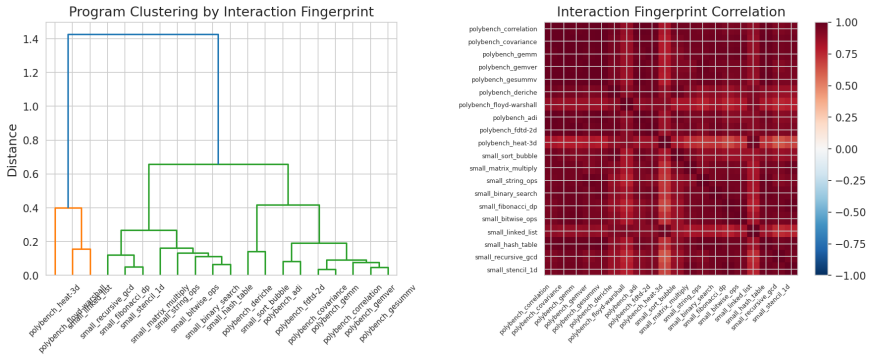

Figure 5: Hierarchical clustering of programs by interaction matrix similarity. Programs from different categories (PolyBench vs. small programs) cluster together based on shared optimization dynamics rather than structural similarity.

Chris Cummins, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somber Joshi, Jiawen Liu, Animesh Jain, and Hugh Leather. CompilerGym: Robust, performant compiler optimization environments for AI research. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2022.

Chaoyi Deng, Jialong Wu, Ningya Feng, Jianmin Wang, and Mingsheng Long. CompilerDream: Learning a compiler world model for general code optimization. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025.

Fabian Fumagalli, Maximilian Muschalik, Patrick Kolpaczki, Eyke Hüllermeier, and Barbara Hammer. KernelSHAP-IQ: Weighted least-square optimization for Shapley interactions. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Michel Grabisch and Marc Roubens. An axiomatic approach to the concept of interaction among players in cooperative games. *International Journal of Game Theory*, 28(4):547–565, 1999.

Michael R. Jantz and Prasad A. Kulkarni. MiCOMP: Mitigating the compiler phase-ordering problem using optimization sub-sequences and machine learning. *ACM Transactions on Architecture and Code Optimization (TACO)*, 14(3), 2017.

Sameer Kulkarni and John Cavazos. Mitigating the compiler optimization phase-ordering problem using machine learning. In *Proceedings of the ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2012.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Riyadh Mammadli, Marija Selakovic, Felix Wolf, and Michael Pradel. Efficient compiler optimization by modeling passes dependence. *CCF Transactions on High Performance Computing*, 2024.

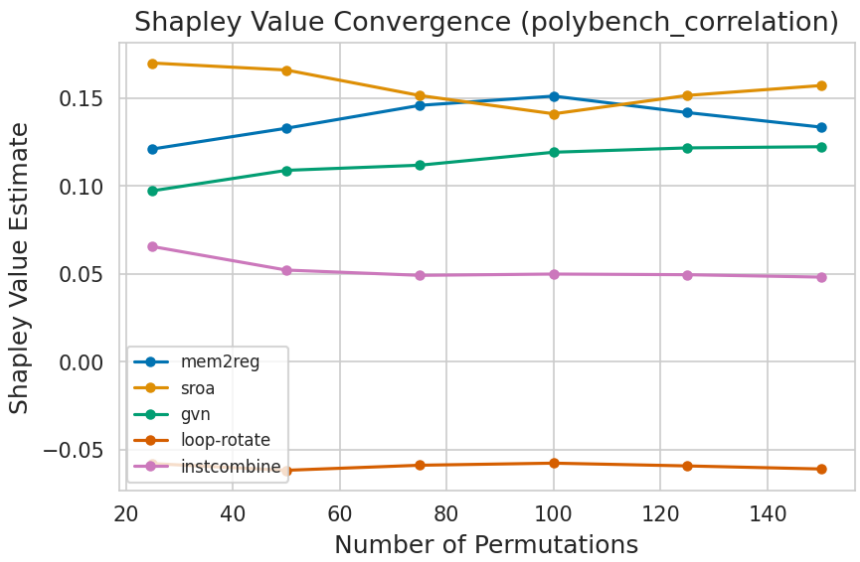

Figure 6: Convergence of Shapley value estimates as a function of Monte Carlo permutations M. Top-ranked passes stabilize within $M = 200$ permutations. Shown for representative programs.

Maximilian Muschalik, Hubert Baniecki, Fabian Fumagalli, et al. shapiq: Shapley interactions for machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Louis-Noël Pouchet. PolyBench/C 4.2: The polyhedral benchmark suite. `http://polybench.sourceforge.net`, 2012.

Lloyd S. Shapley. A value for n-person games. *Contributions to the Theory of Games*, 2(28):307–317, 1953.

Mukund Sundararajan, Kedar Dhamdhere, and Ashish Agarwal. The Shapley Taylor interaction index. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

Mircea Trofin, Yundi Qian, Eugene Brevdo, Zinan Lin, Krzysztof Choromanski, and David Li. MLGO: A machine learning guided compiler optimizations framework. *arXiv:2101.04808*, 2021.

Yu Wang, Hongyu Chen, and Ke Wang. Beyond the phase ordering problem: Finding the globally optimal code w.r.t. optimization phases. *arXiv:2410.03120*, 2024.

Debbie L. Whitfield and Mary Lou Soffa. An approach to ordering optimizing transformations. In *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 1997.